CS 598 PSL Fall 2026

Coding Assignment 4

Due Date: Monday, November 2 (11.59pm)

- Anirudh Eswara
- Ryan Lenea
- Mark Synowiec
- Robert Walker

In this problem we are going to work with the Italian olive oil fatty-acid data of Forina, Armanino, Lanteri, and Tiscornia (1983). The data contain n = 572 authentic olive oils collected in Italy. Each oil is described by the percentage composition of eight fatty acids in the lipid fraction (palmitic, palmitoleic, stearic, oleic, linoleic, linolenic, arachidic, and eicosenoic). Oils come from nine growing areas that nest in three broader regions:

Region	Growing areas	n
South	North-Apulia, South-Apulia, Calabria, Sicily	323
Sardinia	Inland-Sardinia, Coastal-Sardinia	98
North	Umbria, East-Liguria, West-Liguria	151

The scientific question is whether fatty-acid composition can recover the geographic origin of an oil. That is a classic authenticity / chemometrics problem, and a natural setting in which to compare Gaussian discriminant rules with a multinomial logistic model.

The file olive oils.csv can be downloaded from Coursera. Use the eight fatty-acid measurements as predictors. The response is the three-level factor Region (South, Sardinia, North). Do not use Area as a predictor or as the response.

Two notes on the measurements:

- The fatty-acid columns are stored as percentages ×100 (so they sum to about 10,000 rather than 100). Treat them as eight continuous predictors; no further rescaling is required unless a method you use needs it.
- The eight acids are compositional (they describe parts of a whole). For this assignment you do not need a compositional transformation. Do not interpret a coefficient as if that acid could change while the others stayed fixed.

The goal is to compare Discriminant Analysis methods (linear and quadratic) vs. a Logistic/Multinomial approach to classify the three region.

For the questions below, split the data into training and testing (70%–30%) using 598 as a seed. Stratify the split by Region so that all three classes appear in both sets. Use the default training-set class priors.

CS 598 PSL Fall 2026

Coding Assignment 4

Due Date: Monday, November 2 (11.59pm)

- Anirudh Eswara
- Ryan Lenea
- Mark Synowiec
- Robert Walker

In this problem we are going to work with the Italian olive oil fatty-acid data of Forina, Armanino, Lanteri, and Tiscornia (1983). The data contain n = 572 authentic olive oils collected in Italy. Each oil is described by the percentage composition of eight fatty acids in the lipid fraction (palmitic, palmitoleic, stearic, oleic, linoleic, linolenic, arachidic, and eicosenoic). Oils come from nine growing areas that nest in three broader regions:

Region	Growing areas	n
South	North-Apulia, South-Apulia, Calabria, Sicily	323
Sardinia	Inland-Sardinia, Coastal-Sardinia	98
North	Umbria, East-Liguria, West-Liguria	151

The scientific question is whether fatty-acid composition can recover the geographic origin of an oil. That is a classic authenticity / chemometrics problem, and a natural setting in which to compare Gaussian discriminant rules with a multinomial logistic model.

The file olive oils.csv can be downloaded from Coursera. Use the eight fatty-acid measurements as predictors. The response is the three-level factor Region (South, Sardinia, North). Do not use Area as a predictor or as the response.

Two notes on the measurements:

- The fatty-acid columns are stored as percentages ×100 (so they sum to about 10,000 rather than 100). Treat them as eight continuous predictors; no further rescaling is required unless a method you use needs it.
- The eight acids are compositional (they describe parts of a whole). For this assignment you do not need a compositional transformation. Do not interpret a coefficient as if that acid could change while the others stayed fixed.

The goal is to compare Discriminant Analysis methods (linear and quadratic) vs. a Logistic/Multinomial approach to classify the three region.

For the questions below, split the data into training and testing (70%–30%) using 598 as a seed. Stratify the split by Region so that all three classes appear in both sets. Use the default training-set class priors.

In [ ]:
```
import pandas as pd

file_path = "olive_oils.csv"

try:
    df = pd.read_csv(file_path)

    display(df.head())
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

	Region	Area	palmitic	palmitoleic	stearic	oleic	linoleic	linolenic	arachidic	eic
0	South	North-Apulia	1075	75	226	7823	672	36	60	
1	South	North-Apulia	1088	73	224	7709	781	31	61	
2	South	North-Apulia	911	54	246	8113	549	31	63	
3	South	North-Apulia	966	57	240	7952	619	50	78	
4	South	North-Apulia	1051	67	259	7771	672	50	80	

In [ ]:
```
import sklearn

# split data into training and testing (70%–30%) using 598 as a seed
# stratify the split by region
RANDOM_SEED=598

X = df[["palmitic", "palmitoleic", "stearic", "oleic", "linoleic",
"linolenic", "arachidic", "eicosenoic"]]
y = df["Region"]

X_train, X_test, y_train, y_test =
sklearn.model_selection.train_test_split(X, y, test_size=0.3,
random_state=RANDOM_SEED, stratify=y)


# Verify stratification
print("Training Split")
print(len(y_train[y_train == "South"]) / len(y_train))
print(len(y_train[y_train == "North"]) / len(y_train))
print(len(y_train[y_train == "Sardinia"]) / len(y_train))
```

```
print("\nTesting Split")
print(len(y_test[y_test == "South"]) / len(y_test))
print(len(y_test[y_test == "North"]) / len(y_test))
print(len(y_test[y_test == "Sardinia"]) / len(y_test))
```

```
Training Split
0.565
0.265
0.17

Testing Split
0.563953488372093
0.2616279069767442
0.1744186046511628
```

(a) Using the training data fit three models: LDA, QDA and Multinomial (Logistic) regression. Note that here the response has three levels, so a binomial logistic is not applicable.

In [ ]:
```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis as
QDA
from sklearn.linear_model import LogisticRegression

# LDA
lda = LDA(store_covariance=True)
lda.fit(X_train, y_train)

# QDA
qda = QDA(store_covariance=True)
qda.fit(X_train, y_train)

# Multinomial (Logistic) regression
multinom = LogisticRegression(max_iter=5000)
multinom.fit(X_train, y_train)
```

▾ LogisticRegression

▾ Parameters

max_iter	5000
penalty	'deprecated'
C	1.0
l1_ratio	0.0
dual	False
tol	0.0001
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	None
solver	'lbfgs'
verbose	0
warm_start	False
n_jobs	None

▾ Fitted attributes

Name	Type	Value
classes_	ndarray[object](3,)	['North','Sardinia','South']
coef_	ndarray[float64](3, 8)	[[0.01, 0.04, 0.01,...,-0. ,-0. ,...,-0.03, 0.19,-0.18], [0.02, 0.52]]
feature_names_in_	ndarray[object](8,)	['palmitic','palmitoleic','stear 'eicosenoic']
intercept_	ndarray[float64](3,)	[-0.,-0., 0.]
n_features_in_	int	8
n_iter_	ndarray[int32](1,)	[1591]

(b) Report the accuracy for all methods on both training and testing data sets, and prepare confusion matrices for the test predictions. Label the rows and columns by region, not by integer codes. Rows are the true region; columns are the predicted region.

In [ ]:
```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score

def confusion_df(y_true, y_pred, labels):
    # rows = true region, columns = predicted region
    return pd.DataFrame(
        confusion_matrix(y_true, y_pred, labels=labels),
        index=[f"true {l}" for l in labels],
        columns=[f"pred {l}" for l in labels],
    )

# LDA
lda_pred = lda.predict(X_test)
lda_pred_train = lda.predict(X_train)

print("LDA train accuracy:", accuracy_score(y_train, lda_pred_train))
print("LDA test accuracy:", accuracy_score(y_test, lda_pred))
print("LDA test confusion matrix (rows = true, columns = predicted):")
display(confusion_df(y_test, lda_pred, lda.classes_))

# QDA
qda_pred = qda.predict(X_test)
qda_pred_train = qda.predict(X_train)

print("QDA train accuracy:", accuracy_score(y_train, qda_pred_train))
print("QDA test accuracy:", accuracy_score(y_test, qda_pred))
print("QDA test confusion matrix (rows = true, columns = predicted):")
display(confusion_df(y_test, qda_pred, qda.classes_))

# Multinomial (Logistic) regression
multinom_pred = multinom.predict(X_test)
multinom_pred_train = multinom.predict(X_train)

print("Multinomial (Logistic) regression train accuracy:",
accuracy_score(y_train, multinom_pred_train))
print("Multinomial (Logistic) regression test accuracy:",
accuracy_score(y_test, multinom_pred))
print("Multinomial (Logistic) regression test confusion matrix (rows =
true, columns = predicted):")
display(confusion_df(y_test, multinom_pred, multinom.classes_))
```

```
LDA train accuracy: 0.9875
LDA test accuracy: 1.0
LDA test confusion matrix (rows = true, columns = predicted):
```

	pred North	pred Sardinia	pred South
true North	45	0	0
true Sardinia	0	30	0
true South	0	0	97

```
QDA train accuracy: 1.0
QDA test accuracy: 1.0
QDA test confusion matrix (rows = true, columns = predicted):
```

	pred North	pred Sardinia	pred South
true North	45	0	0
true Sardinia	0	30	0
true South	0	0	97

```
Multinomial (Logistic) regression train accuracy: 1.0
Multinomial (Logistic) regression test accuracy: 1.0
Multinomial (Logistic) regression test confusion matrix (rows = true,
columns = predicted):
```

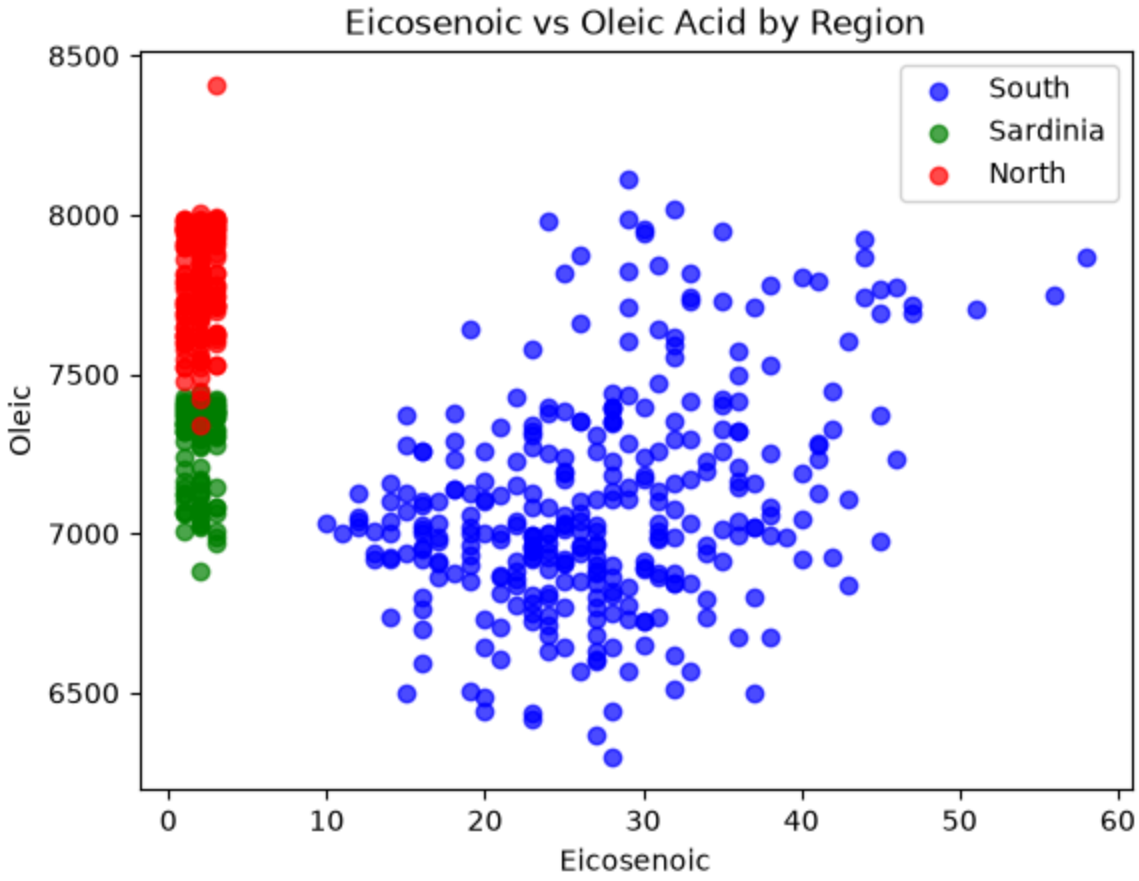
	pred North	pred Sardinia	pred South
true North	45	0	0
true Sardinia	0	30	0
true South	0	0	97

(c) Produce one scatterplot of two fatty acids with points colored by Region. Put eicosenoic on one axis (the other axis can be oleic or linoleic). Briefly state what the plot suggests about class separation before you interpret the fitted models.

In [ ]:
```
import matplotlib.pyplot as plot

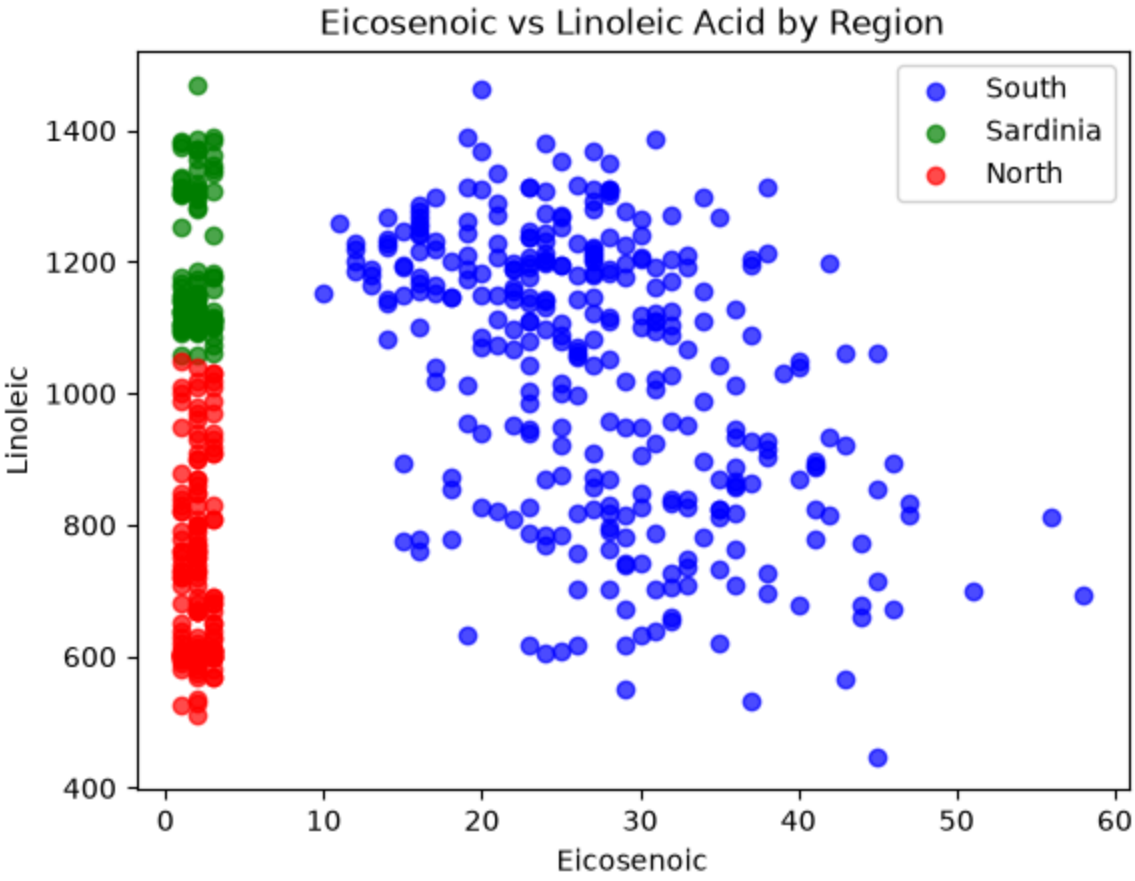
plot.scatter(df[df["Region"] == "South"]["eicosenoic"], df[df["Region"] ==
"South"]["oleic"], color="blue", alpha=0.7, label = "South")
plot.scatter(df[df["Region"] == "Sardinia"]["eicosenoic"], df[df["Region"]
== "Sardinia"]["oleic"], color="green", alpha=0.7, label = "Sardinia")
plot.scatter(df[df["Region"] == "North"]["eicosenoic"], df[df["Region"] ==
"North"]["oleic"], color="red", alpha=0.7, label = "North")
plot.xlabel("Eicosenoic")
plot.ylabel("Oleic")
plot.title("Eicosenoic vs Oleic Acid by Region")
plot.legend()
```

```
<matplotlib.legend.Legend at 0x11ca349e0>
```



In [11]:
```
plot.scatter(df[df["Region"] == "South"]["eicosenoic"], df[df["Region"] ==
"South"]["linoleic"], color="blue", alpha=0.7, label = "South")
plot.scatter(df[df["Region"] == "Sardinia"]["eicosenoic"], df[df["Region"]
== "Sardinia"]["linoleic"], color="green", alpha=0.7, label = "Sardinia")
plot.scatter(df[df["Region"] == "North"]["eicosenoic"], df[df["Region"] ==
"North"]["linoleic"], color="red", alpha=0.7, label = "North")
plot.xlabel("Eicosenoic")
plot.ylabel("Linoleic")
plot.title("Eicosenoic vs Linoleic Acid by Region")
plot.legend()
```

Out[11]:
```
<matplotlib.legend.Legend at 0x11786da90>
```



Scatterplot Interpretation

Eicosenoic acid clearly delineates the South region from the other two, but between the North and Sardinia regions, there is little difference in Eicosenoic. Both Linoleic and Oleic acids do well, however, to delineate between the North and Sardinia regions, with very little overlap between the two.

(d) Comment on the results. Address all of the following:

- Is there a model that seems to perform better overall on the test set?
- Which model misclassified more observations?
- Which region is classified most / least accurately, and which pair of regions is most often confused?
- Does QDA improve on LDA here, or does the extra covariance structure look like overfit? Relate this to the class sizes and to part (c).

The Sardinia test slice is small (on the order of 30 oils). Comment on the direction of the errors, not on a one- or two-oil swing in accuracy.

Remark: The results might be different between Python and R. That is expected and ok.

All three models (LDA, QDA and Multinomial (Logistic) regression) achieved an accuracy of 100% on the test set with zero misclassifications. All the regions were

classified equally accurately with an accuracy of 100%. Therefore, no pair of regions were confused. Both QDA and LDA perfectly classify on the test data; so QDA does NOT improve on LDA. Given that both QDA and LDA achieved an accuracy of 100% on the test set, we do not have evidence that the extra covariance structure looks like overfit. Training accuracy was 98.75% for LDA (5 of 400 training oils misclassified) and 100% for both QDA and multinomial regression, so the training and test results are consistent and there is no train/test gap indicating overfit. LDA's test accuracy being slightly higher than its training accuracy is expected from a single split and is not meaningful. Since *p* is not large relative to n_k, the class covariances are not singular and are unlikely to be noisy. Since the scatterplots in part (c) show little overlap between the classes in the feature space, the LDA is preferred over the QDA as it has fewer parameters. Since the three models achieved an accuracy of 100% on the test set, there is no direction of the errors.